

Sheel Shah

Computer Science Student

shs143@ucsd.edu | +1 (510) 556 7792 | Milpitas, CA | sheel.dev | github.com/shahxsheel | linkedin.com/in/shahxsheel

SUMMARY

Computer science student who spent 2026 shipping real-time ML on constrained hardware - a Raspberry Pi driver-monitoring pipeline running 15+ FPS at sub-70ms latency, a CAN-bus robotic arm control stack ported to macOS from scratch, and context-management infrastructure for AI coding agents. Works across embedded firmware, Python inference, and production TypeScript frontends.

EXPERIENCE

Research Intern - Infineon Technologies (In Partnership with De Anza College)

Winter 2026 - Winter 2026

Selected from a pool of 300+ CS students into a 5-person research cohort. Faculty advisor: Prof. Saied Rafati.

- Owned the inference pipeline for ARGUS (Autonomous Real-time Guardian for Ubiquitous Safety), a driver-monitoring safety system, hitting 15+ FPS at sub-70ms latency and a 63% lower-latency result versus the initial implementation on Raspberry Pi hardware.
- Built the detection stack from scratch: YOLO26s object detection via raw ncnn and MediaPipe FaceLandmarker blendshape eye-closure detection, split across two processes communicating over ZeroMQ so camera capture never blocks on inference latency.
- Eliminated the torch/ultralytics runtime dependency on Raspberry Pi 4B - cutting about 150MB and removing an "Illegal instruction" crash class - by pre-exporting the model to NCNN and shipping it via a release artifact.
- Engineered a driver-lock and risk-scoring engine: MOSSE/KCF tracker lock-on, rolling PERCLOS drowsiness windows, head-pose off-road gaze duration, and phone-presence timing, with three tunable risk policies and cooldown-gated alerting.
- Exposed about 40 documented environment variables for camera, detection cadence, and risk thresholds; wrote a pytest suite covering driver lock, face/eye caching, risk engine, and scheduling.
- Contributed gyroscope sensor handling to the team monorepo (Infineon ModusToolbox C firmware, Swift/SwiftUI iOS companion, Supabase/PostgreSQL backend).
- System premiered publicly at STEM Zone 2026 (Santa Clara Convention Center) - the only De Anza team selected to present. Research paper published.

Teaching Assistant, Mathematics & Computer Science - De Anza College

Fall 2025 - Spring 2026

- Supported 90+ students across Intro C++, Intermediate C++, Data Structures & Algorithms, and Calculus I-III, holding 12 weekly office hours.
- Re-taught missed material one-on-one for students returning from absences; tracked attendance and ran individual follow-up to keep at-risk students on pace.

Finance Intern - De Anza College Student Government

Winter 2025 - Fall 2025

- Built the full presentation for the 2025-26 budget (about \$1.4M, representing about 18,000 students) delivered to the Foothill-De Anza Community College District Board of Trustees.
- Ran an independent campus-wide research project, surveying 100+ students on program participation, and presented findings to the Student Government Finance team.

Volunteer - Jain Center of Northern California (JCNC)

Sept 2024 - Present | Milpitas, CA

- Supported fundraising operations for a temple renovation campaign (Summer 2025); ongoing general volunteer.

EDUCATION

B.S., Computer Science, Jacobs School of Engineering (Eleanor Roosevelt College) - University of California, San Diego

Expected Spring 2028

A.S., Computer Science; Mathematics - De Anza College

Fall 2024 - Spring 2026 | 3.81

Courses: Data Structures & Algorithms, Intermediate C++, Calculus I-III, Human-Computer Interaction

SKILLS

- **Languages:** Python, TypeScript, JavaScript, C++, C, Swift, Dart, Rust, SQL
- **AI / ML:** YOLO (v8n, 26s), ncn, MediaPipe, OpenCV, ONNX, PyTorch (export), SARIMA, Claude API, Gemini, DeepSeek, Groq, ElevenLabs (TTS + Conversational AI), LLM-as-judge evaluation, prompt/context engineering
- **Backend & Infra:** FastAPI, Node.js, WebSockets, ZeroMQ, REST API design, Supabase, PostgreSQL, SQLite, Docker-adjacent deploy (DigitalOcean App Platform, Render), GitHub Actions CI/CD, uv, Bun
- **Frontend:** React 18, Svelte, Vite, Tailwind CSS, Flutter, SwiftUI, Tauri, Leaflet, HTML5 Canvas
- **Embedded & Robotics:** Infineon ModusToolbox, TuyaOpen (ARM Cortex-M33), LVGL v9, Raspberry Pi 4B/5, CAN bus, gs_usb, SLCAN, Piper robotic arm SDK, Orbbec depth cameras, BLE GATT, PiCamera2
- **Practices:** Git feature-branch workflows and PR review, pytest, real-time systems tuning, latency/throughput profiling, technical documentation, research writing

PROJECTS

Accordion

Present

Context-window management for AI coding agents

- Renders an AI agent's entire context window as a live grid of foldable blocks - users fold, unfold, and pin segments reversibly instead of losing information to lossy compaction or sliding-window truncation.
- Built a pluggable tiered-relevance WebSocket Conductor (an automatic fold/unfold strategy), then shipped same-day fixes for group-sweep behavior, a zero-saving compression edge case, and a first-tier relevance blindness bug.
- Shipped a merged PR making the system prompt a first-class, non-foldable config dial: thousands of tokens that had been invisible in the UI and untracked in every token budget. Wired capture through `ExtensionContext.getSystemPrompt()` at a hook point verified against upstream source to avoid a race condition, corrected a documented token-calibration gap, bumped the wire protocol with round-trip serialization, and added a new Svelte UI panel. 746/746 tests passing, type-check clean.
- Authored the project's GitHub Actions CI/CD for Windows release-installer builds, plus the fix for push-triggered `workflow_dispatch` branch registration.
- Contributed continuously for over a month past initial build; 200+ GitHub stars.

Technologies: *TypeScript, Svelte, Rust (Tauri), Python, WebSocket, GitHub Actions*

Cura

Present

AI-powered robotic feeding assistant for hospital patients

- Controls a Piper robotic arm over CAN bus to pick up and deliver food/water to bedridden patients, coordinated by a FastAPI server driving a state-machine orchestrator (IDLE to APPROACHING to GRASPING to LIFTING to DELIVERING to DRINKING to RETRACTING).
- Built macOS CAN-bus support from scratch - the Piper SDK toolchain is Linux-only. Added `gs_usb` transport, diagnosed and fixed a reconnect failure and a macOS segfault, switched hardware detection from `system_profiler` to `ioreg` for non-interactive scripting, and implemented SLCAN as an alternate path.
- Wrote the vision-to-arm bridge, mapping MediaPipe 2D landmarks to 3D real-world coordinates and integrating Orbbec depth-camera input (`pyorbbecsdk2`) for camera-guided positioning.
- Debugged a real-time firmware constraint under deadline - arm motion required streaming `EndPoseCtrl` at 100Hz - and built the full project scaffold spanning arm control, vision, interface, and orchestrator layers.
- Recognized by event organizers for cross-team debugging support on computer-vision issues.

Technologies: *Python, piper_sdk, CAN bus, gs_usb, SLCAN, FastAPI, MediaPipe, YOLOv8n, Orbbec SDK, LVGL/TuyaOpen (T5AI, ARM Cortex-M33)*

Temper

Present

Environment-level evaluation and auto-remediation for AI agent deployments

- Tests the harness wrapped around a model - system prompt, skill files, tool definitions - rather than the model itself, then auto-generates patches. Runs identical prompts against a bare-model baseline and the full harness and computes a per-dimension delta across instruction adherence, tool-call accuracy, output format, skill trigger accuracy, latency, and error recovery.
- Built the live evaluation dashboard and multi-session room infrastructure, plus a coding-benchmark question mode, and drove the project through DigitalOcean App Platform deployment.
- Carried the rooms/dashboard feature through four rounds of PR review under time pressure, resolving P0/P1/P2 issues including room authentication and dashboard metric correctness.
- Benchmark case: tool-call accuracy recovered 31 to 79 and instruction adherence 44 to 82 after auto-patching.

Technologies: *Python, FastAPI, Gemini 2.5 Flash, DeepSeek, DigitalOcean, LLM-as-judge*

Clear Dispatch (SIGNAL)

Present

Human-in-the-loop 911 dispatch support for wildfire surge events

- Four coordinated AI agents work behind a human dispatcher - MONITOR (sliding-window call-rate watcher), TRIAGE (Claude Haiku severity and vulnerable-caller classification), RESOURCE (Haversine nearest-unit routing with a hard human-confirmation gate for heavy assets), and RELAY (Claude Haiku + ElevenLabs TTS dispatcher briefings) - under a strict design rule that AI never speaks directly to a caller.
- Built the entire operator dashboard from scratch: React 18 + TypeScript + Vite + Tailwind, an auto-reconnecting WebSocket hook driving a pure useReducer over 8 real-time message types, with live agent-status cards, severity-badged call queue, Leaflet map rendering real GeoJSON fire perimeters and unit markers, override controls, hold modal, briefing panel, and audit trail.
- Shipped the live-demo centerpiece: real-time call transcription, a conversational voice agent (ElevenLabs Conversational AI + GPS), and a phone-scannable QR page letting an observer call in live from their own device.
- Set up GitHub Actions CI, added a sanitized in-memory logging endpoint, and resolved a multi-branch merge conflict between a UI redesign and a feature branch.
- Modeled on the 2024 Park Fire (400,000+ acres) with 8 real Yolo County vulnerability zones and CAL FIRE-realistic resource types.

Technologies: React 18, TypeScript, Vite, Tailwind, Python 3.11, FastAPI, WebSocket, Claude API, ElevenLabs, Leaflet

Echo

Present

Offline emergency alert mesh over Bluetooth

- Flutter app relaying National Weather Service alerts phone-to-phone over BLE with zero internet or cell service after the first device receives an alert; every receiving device automatically becomes a relay. Measured 7 hops in under 3 seconds.
- Chunks alert JSON across BLE GATT characteristic reads, reassembles on the receiver, persists to SQLite, and re-advertises - with an alertIdHash dedup check preventing relay loops and a hop counter showing propagation depth.
- Implemented the hop-counter relay logic, fixed the NWS API integration, resolved iOS integration issues, and built the offline multilingual pipeline (verified rendering alerts in Hindi and English).
- Ran the mesh loop in a background isolate (flutter_background_service) on an autonomous 5-minute scan-dedupe-download-persist-notify cycle, kept separate from the main engine because the native GATT-server MethodChannel is main-thread-only.
- Tested against physical Raspberry Pi relay nodes via a companion Python CLI broadcaster.

Technologies: Dart, Flutter, BLE GATT, SQLite, Python, NWS API

Wizlet

Present

Real-time multiplayer study game

- Turns uploaded lecture-note PDFs into a head-to-head quiz battle: a Groq-backed microservice converts a PDF into difficulty-tagged multiple-choice questions, and two players fight using spells tied to answer correctness.
- Built a three-service architecture - canvas-based Vite/TypeScript battle client, a Node WebSocket game server with room-code-based matchmaking and waiting/playing/finished state, and an Express-style PDF-to-questions REST service.
- Fixed cross-player question synchronization (both players must answer before advancing), reworked the combat loop with spell selection and answer feedback, added an end-game summary and a play-again flow supporting a fresh PDF without leaving the room.
- Shipped with deployment configuration and architecture documentation; deployed via Render. MIT licensed.

Technologies: TypeScript, Vite, Node.js, WebSocket, GeminiAPI

Halo

Present

Gesture-controlled photo booth

- Real-time hand/pose gesture detection driving live "magical aura" video effects - four gestures each mapped to a distinct effect (halo rings, heart sprites, branching lightning, fingertip particles).
- Targets about 30 FPS at 720p using MediaPipe landmark detection, with a about 1-second consistency requirement before locking a gesture to prevent flicker, plus a manual fallback if the camera is unavailable.
- Built the capture pipeline: watermarked PNG export, shareable QR code generation, canvas DPR scaling fixes, capture timers, and a live performance HUD showing camera FPS and resolution.
- Validated the gesture-classification algorithm against a separate Python reference implementation with Pydantic schemas and unit tests.

Technologies: TypeScript, Vite, MediaPipe, Python, Pydantic

Aura

Present

Description

Brain-controlled robotic manipulation through high-level intent.

- Built a real-time BCI-to-robotics pipeline using EEG signal processing, machine learning, computer vision, and robotic control.
- Implemented P300-based object selection with 82.8% accuracy and Motor Imagery classification for grasp and release commands.
- Integrated OpenBCI Cyton EEG acquisition at 125 Hz with multiprocessing for real-time signal processing and ML inference.
- Used ArUco marker tracking and planar homography to localize objects and target locations.
- Applied inverse kinematics to control a custom 3-axis robotic arm.

Technologies: Python, OpenBCI, pyOpenBCI, scikit-learn, OpenCV, ArUco, IKPy, Tkinter, multiprocessing, Arduino, EEG, P300, Motor Imagery, FBCSP, LDA, Random Forest

Mocha

Present

A self-hosted Proxmox platform serving file, photo, and media services to a household daily, with a custom control plane for remote administration.

- Serve Nextcloud, Immich, and Jellyfin from a Proxmox hypervisor spanning a mini PC and a Raspberry Pi 5, on RAID/ZFS-backed storage
- Split ingress into two paths so the management plane has no public attack surface: a Cloudflare Tunnel for services that must be reachable, Tailscale for everything private
- Built a FastAPI control panel for the operations run most often, storage allocation and container startup, bound only to the Tailscale interface
- Wrote udev rules so external storage is detected on hotplug instead of requiring manual mount steps after every reconnect
- Restored service after failures at three layers: container restart and orchestration faults, a disk and filesystem incident that risked stored data, and routing and DNS faults where services were up but unreachable

Technologies: Proxmox, Linux, Docker, ZFS, RAID, nginx, Cloudflare Tunnel, Tailscale, FastAPI, Python, udev, Raspberry Pi, Ubuntu

SimplyApply

Present

Open-source, self-hosted internship tracker and resume tailoring platform with a code-enforced guardrail against fabricated experience.

- Architected a full-stack resume tailoring platform (Next.js/React + FastAPI) that regenerates ATS-safe resumes from structured JSON Resume data instead of editing a file in place, tailoring each apply to a specific job posting
- Built a deterministic no-fabrication guardrail that cross-validates every LLM-tailored resume against the user's base resume, checking employers, titles, dates, and metrics, with automatic retry-on-violation and fallback to the original resume; verified by 85 automated pytest tests
- Integrated 4 pluggable LLM provider adapters (Anthropic, OpenAI-compatible, Ollama, OpenRouter) behind a common interface, plus Greenhouse and Ashby job board connectors and a GitHub-sourced internship tracker with 15-minute caching and stale-cache fallback.
- Shipped a single-page resume rendering pipeline producing both PDF (reportlab, scale-to-fit) and DOCX (python-docx) outputs, plus an automated ATS-match review that scores tailored resumes against the target job description

Technologies: Next.js, React, TypeScript, Tailwind CSS, FastAPI, Python, SQLAlchemy, SQLite, Pydantic, Anthropic API, OpenAI API, Ollama, pytest, Vitest, Docker, OpenRouter

Dino Run Blink

Present

Blink-controlled gameplay using real-time brain-computer interface signal processing.

- Built a real-time EOG blink detection system using signals captured from a MUSE 2 BCI headset at 256 Hz.
- Processed AF7 and AF8 sensor data using spatial averaging, Butterworth bandpass filtering, notch filtering, and RMS-based feature extraction.
- Implemented threshold-based blink detection to translate physiological signals into real-time game controls.
- Integrated Lab Streaming Layer (LSL) for live EEG/EOG data streaming and pynput for simulated keyboard input.
- Added calibration, configurable detection parameters, and real-time RMS visualization for tuning and debugging.

Technologies: Python, MUSE 2, EOG, EEG, Lab Streaming Layer, pylsl, SciPy, NumPy, Matplotlib, pynput, Butterworth Filter, RMS Signal Processing, Petalmetrics